

Fiori Elements 開發課程 10：部署 (Deployment)

環境：BTP ABAP Environment Trial (套件 **ZRAPCLOUD**，沿用 fe01 的 **fe01_connection_test** 前端專案)

Lecture

這一課要解決的問題

fe01 ~ fe09 一路都是「本機 **npm start** 透過 Fiori Tools Proxy 連真實後端」——這只是開發時的預覽，App 的程式碼始終留在你電腦的硬碟上，其他人 (包含真正的業務使用者) 完全看不到、用不到。這一課要做的是部署 (**Deploy**)：把 App 打包上傳，變成 ABAP 系統裡一個真正的持久物件，之後任何有權限的人都能透過瀏覽器直接開啟。

沿用 fe01 就建立、fe02 ~ fe09 一路疊加修改的 **fe01_connection_test** 專案 (指向 rc08 的 **ZI_RC05_NOTE**，有 Draft、有 Create/Edit 按鈕的完整 App)，把它部署成一個正式的 SAPUI5 ABAP Repository 物件。

架構對照：跟 React 部署模型比起來，JS 到底上傳去哪？Fiori Launchpad 又是什麼角色？

Fiori Elements / SAPUI5 本質上也是一個純前端 JavaScript 框架，**npm run deploy** 背後一樣先做一次 build (**ui5 build**，把 **webapp/** 打包成靜態的 HTML/JS/CSS/JSON)，這點跟 React 完全一樣。差別在打包完之後，這份靜態檔案搬運到哪裡、由誰負責 **serve** 出去：

	React 常見做法	Fiori Elements (這次用的 ABAP 部署目標)
打包結果	靜態 HTML/JS/CSS bundle	同樣是靜態 HTML/JS/CSS/JSON bundle (manifest.json 、 Component.js)
搬運方式	scp / CI 上傳到某台主機，或推到 S3 / Vercel	打包成 zip ，透過 /UI5/ABAP_REPOSITORY_SRV 這支 OData Service 上傳—— ui5-deploy.yaml 裡的 target.authenticationType: reentranceTicket 就是在跟這支 Service 做身分驗證
存放位置	一般檔案系統 或 物件儲存 (S3)	存進 ABAP 系統自己的 Repository ，變成一個叫 BSP Application 的物件 (ZFE10_NOTE_APP)——本質上是 ABAP 系統內建的一個「小型檔案樹」容器，跟你在 Eclipse 裡看到的 Class / CDS View 是同一類「Repository 物件」，只是內容物是 HTML/JS 而不是 ABAP 原始碼，所以才需要跟其他 ABAP 物件一樣指定套件 (ZRAPCLOUD)
誰負責 serve 這些靜態檔案	Nginx / Vercel / CloudFront 這類通用 Web Server	ABAP 系統自己內建的 HTTP Server ，叫 ICM (Internet Communication Manager，SAP Kernel 內建模組)。部署時額外建立的 ICF Node ，作用是「把某個 URL 路徑掛到這個 BSP Application 上」——概念上等同 Nginx 的一條 location 設定，只是這個 Web Server 內建在 ABAP Kernel 裡，不是外部另外裝的

結論：這次的 **JavaScript** 沒有跑到另一台獨立的 **Web Server** 上，而是直接存進了 ABAP 系統的資料庫 (透過 Repository 物件包裝)，再由這個 ABAP 系統自帶的 HTTP Server (ICM/ICF) 直接對外 **serve** 出去。

Fiori Launchpad 不是「連到 Web Server」的那一層，它本身也是被同樣方式 **serve** 出來的一個 **App**——這是最容易誤解的地方。Fiori Launchpad (FLP) 不是 gateway、不是 reverse proxy，它本質上也只是另一個 **SAPUI5 App** (一個「Shell」殼應用)，跟 **ZFE10_NOTE_APP** 一樣被 ICM/ICF serve 出來，只是 SAP 標準就內建好了。它的角色比較接近「App 目錄首頁」：

- 它讀取的是 **Launchpad App Descriptor Items** (fe11 才會建立：Tile + Semantic Object/Action + Target Mapping)，這些資料告訴 FLP「使用者點這個 Tile → 要載入哪個 App (哪個 **manifest.json**) → 用什麼參數」
- 使用者點 Tile 時，FLP 是在瀏覽器端動態把目標 **App** 當成一個 **UI5 Component** 載進自己的殼子裡 (概念上類似 React 的 micro-frontend / module federation)，不是幫你轉發 HTTP 請求
- 所以就算完全不設定 FLP Tile (就像本課這次的 **ZFE10_NOTE_APP**)，部署完成的 App 本來就已經是一個獨立可用的網址 (**/sap/bc/ui5-ui5/...**)，可以直接貼給人用——本課「驗證方式」那一步就是這樣驗收的。FLP 純粹是「不用每個人各自記網址，改成一個入口網站、點圖示進去」的方便層，不是必要的傳輸路徑

一張圖整理兩者差異：

React 典型架構：

瀏覽器 → CDN/Nginx (靜態檔案，跟後端 API 主機通常是分開的) → (另外呼叫) → API Server

這次 Fiori Elements (ABAP 部署目標)：

瀏覽器 → ICM/ICF (ABAP 系統內建 HTTP Server，serve 部署的 BSP App 靜態檔案)

↓ (同一個 ABAP 系統裡)

OData Service (ZRC08_SB，RAP 後端)

Fiori Launchpad (另一個被同樣方式 serve 出來的 UI5 App)

↓ (瀏覽器端動態載入)

部署的 App (ZFE10_NOTE_APP) 當成 Component 掛進來

額外補充：**fiori add deploy-config** 精靈其實有兩種部署目標可選——這次選的 **ABAP** (靜態檔案跟後端 OData Service 住在同一個 ABAP 系統裡，這一課走的路線)；另一種是 **Cloud Foundry** (部署到 BTP 的 HTML5 Application Repository，一個真正獨立於 ABAP 系統之外的靜態檔案託管服務，中間還會透過一個 Approuter 做路由，更接近典型「前後端分離、各自獨立主機」的 React 部署模型)。這一課用 ABAP 目標是因為後端本來就是 ABAP RAP，兩者放一起最直接；Cloud Foundry 目標留給之後有需要時再探索。

部署會在後端建立什麼 (查證官方文件 **sap-help-1d9deef79d7d4936850b2d6343206ec8**，非猜測)

部署完成後，以下物件會自動建立並記錄在你選的傳輸請求裡：

物件	說明
BSP Application (Business Server Page)	存放 App 的程式碼本體 (打包後的 webapp/ 內容)
MIME Repository 資料夾	存放圖示等 MIME 物件 (Eclipse ADT 看不到，只在傳輸請求清單裡)

物件	說明
ICF Node	讓瀏覽器可以透過 HTTP 存取這個 BSP App (Eclipse ADT 看不到，只在傳輸請求清單裡)
Fiori Launchpad App Descriptor Items	如果 <code>manifest.json</code> 裡有設定 Launchpad Tile / Intent，會一併生成 (這一課還沒設定，留給 fe11)

這代表部署不是「上傳一份檔案」這麼單純——是同時建立四種不同性質的 ABAP Repository 物件，其中兩種 (MIME 資料夾、ICF Node) 連 Eclipse ADT 都看不到，只能透過傳輸請求清單間接確認存在。查詢 BSP App 本身可以用 CDS View `I_CustABAPObjDirectoryEntry` (官方文件明講的查詢入口，[TADIR](#) 的客戶物件導向視圖)。

⚠ BSP Application 到底是什麼？——「借殼」用途，不是真的在跑伺服器端網頁邏輯

BSP = Business Server Page，是 ABAP 系統裡一種很古早的網頁技術 (約 2001 年出現，概念上跟同時代的 JSP / ASP 一樣：用伺服器端邏輯動態產生 HTML)。但這次部署用到的 BSP Application，沒有真的在用「伺服器端渲染」這套機制，只是借用了它的儲存容器 (**Repository** 物件型別)：

- 它本質上是 ABAP 系統裡的一個「檔案容器」物件——跟 Class、CDS View 一樣掛在套件底下 (`ZFE10_NOTE_APP` 就是這個物件的名稱)，只是裡面裝的不是 ABAP 原始碼，而是一整棵檔案樹 (`index.html`、`Component.js`、`manifest.json`、圖示.....)，概念上很接近一個掛在資料庫裡的資料夾
- 對純前端 **UI5 App** 這個用途來說，它完全不執行任何伺服器端 **ABAP** 邏輯——傳統 BSP 頁面是「ABAP 程式碼混著 HTML 標籤、由伺服器算出最終網頁」，但部署的 Fiori Elements App 是 100% 純 JavaScript，所有邏輯都在瀏覽器端跑；BSP Application 這個容器純粹是「存放 + 提供下載」，沒有真的用到它原本的伺服器端渲染能力
- 為什麼借用這個舊技術：SAPUI5 / Fiori 出現時 (約 2012 ~ 2013 年)，ABAP 系統裡剛好已經有 BSP Application 這種「能存放靜態網頁資源、又能透過 ICF 掛 URL 對外服務」的現成容器，SAP 直接沿用當作「SAPUI5 ABAP Repository」的底層儲存機制，不用另外發明新的物件型別
- 對外服務走專屬的 **ICF Handler**：雖然容器沿用 BSP 技術，實際 serve 給瀏覽器的路徑是走一個專門為 UI5 App 優化過的 Handler (`/sap/bc/ui5-ui5/...`)，不是傳統 BSP 頁面用的 `/sap/bc/bsp/...` 路徑，兩者在 ICF 裡是不同的服務端點

一句話：BSP Application 在這裡只是「借殼」——用一個原本設計給伺服器端網頁技術用的容器物件型別，裝這個純前端 JavaScript App 的靜態檔案，跟真正的「Business Server Page 動態網頁邏輯」無關。

ui5-deploy.yaml 關鍵欄位

Fiori: Add Deployment Configuration 精靈跑完會產生 `ui5-deploy.yaml`。下面是這次實際在 `fe01_connection_test` 產生的真實內容 (不是官方文件範例，是這一課端對端跑出來的結果，跟文件範例有落差、以這份實測為準)：

```
# yaml-language-server: $schema=https://sap.github.io/ui5-tooling/schema/ui5.yaml.json

specVersion: "4.0"
metadata:
  name: fe01connectiontest
```

```

type: application
builder:
  resources:
    excludes:
      - /test/**
      - /localService/**
  customTasks:
    - name: deploy-to-abap
      afterTask: generateCachebusterInfo
      configuration:
        target:
          url: https://xxxxxxxx.abap.ap21.hana.ondemand.com
          authenticationType: reentranceTicket # SAML support for vscode
        app:
          name: ZFE10_NOTE_APP
          description: Deploy fe10 Note App
          package: ZRAPCLOUD
          transport: ''
        exclude:
          - /test/
          - /localService/

```

逐項說明：

欄位	意義
<code>target.url</code>	跟 <code>ui5.yaml</code> 裡 <code>fiori-tools-proxy</code> 用的同一個 API 網域 (<code>*.abap.ap21.hana.ondemand.com</code> · fe01 已踩過「不能用 <code>*.abap-web.*</code> 」這個坑)
<code>target.authenticationType</code>	<code>reentranceTicket</code> ——因為選的是已經存過的系統連線 (跟 fe01 的 Generator 精靈同一組已儲存系統) ，精靈自動帶對，不用手動改
<code>app.name / description / package / transport</code>	對應精靈裡填的 SAPUI5 ABAP Repository / Deployment Description / Package / Transport Request ； <code>transport: ''</code> (空字串) 代表 <code>ZRAPCLOUD</code> 這個套件確實不需要傳輸請求 (呼應第 9 節就記錄過「這個套件從未要求過傳輸請求」)
<code>exclude</code>	部署時 不要 打包進去的檔案 (這裡排除 <code>/test/</code> / <code>/localService/</code> ，開發用的假資料 / 測試檔案沒有必要上傳到正式環境)
<code>builder.resources.excludes</code>	這是 build (ui5 build) 階段 的排除清單，跟上面 <code>exclude</code> (部署上傳階段的排除清單) 是兩個獨立設定，剛好這次內容重疊但語意層次不同——build 排除是「打包時就不產生這些檔案」，deploy 排除是「打包完了、但上傳時跳過某些已經打包好的檔案」

⚠ 跟本課一開始查證官方 **ux-ui5-tooling README** 範例的落差 (如實記錄，不是筆誤) ：README 文件描述的通用範例是 `target.scp: true` (一個獨立布林值旗標，官方原文：「若目標系統是 SAP BTP 的 ABAP

Environment，這個參數必須設成 `true`」) + 獨立的 `credentials.authenticationType` 區塊；但這次精靈針對「選擇已存過的 `reentranceTicket` BTP 系統連線」這個情境，實際產生的檔案完全沒有 `scp` 欄位，`authenticationType` 也是直接放在 `target` 底下，不是獨立的 `credentials` 區塊。目前判斷：`authenticationType: reentranceTicket` 這個值本身就已經明確告訴部署工具「這是 BTP ABAP Environment」，`scp` 這個額外旗標可能只在用 **Basic Auth** 手動連 BTP 系統（工具本身無法從認證方式判斷目標系統類型）時才需要——但這只是合理推測，沒有進一步查證版本差異的根本原因，照精靈實際產生的內容操作就對了，不用手動加 `scp` 欄位。

VS Code Step by Step：建立部署設定

1. **Ctrl+Shift+P** → **Fiori: Add Deployment Configuration**——不用先手動開啟專案，這個指令直接下就好
2. 會先跳出「**Select a Fiori tools application**」，讓你從整個工作區（workspace）裡選要設定部署的專案——工作區裡如果同時存在多個 Fiori 專案（`fe01_connection_test` / `fe06.travel.ext` / `fe08taskprojection` / `fe09tasksummary`.....），這一步就是在選對象，這一課選 `fe01_connection_test`（跟 `fe04` 講義記錄過的 Page Map 精靈是同一種行為模式：先選專案，才進到後續設定畫面）
3. **Target**：選 **ABAP**
4. **Select Target System**：選既有的 **TRL**（跟 `fe01` Generator 精靈用的同一個已儲存系統，不用重新輸入 URL / 重新登入）
5. **Enter client**：留空使用預設值（精靈通常會自動帶出正確的 client）
6. **SAPUI5 ABAP Repository**：填 **ZFE10_NOTE_APP**

> 這個名稱是什麼？這不是查詢既有物件、也不是系統自動生成的名稱，是你在這裡自己決定、之後才會被建立出來的技術名稱——目前系統裡還沒有這個物件。填了之後，這個名字會同時變成：①存放程式碼的 **BSP Application** 物件名稱、②部署後存取網址的一部分（例如 `/sap/bc/ui5-ui5/sap/zfe10_note_app/index.html`，小寫）、③之後在 `I_CustABAPObjDirectoryEntry` 查得到的 `obj_name`。之後如果重新部署，同一個名稱 = 更新同一個物件，換一個名稱 = 變成建立全新物件。> > 命名規則：**Z** 開頭是必要的（客戶命名空間，這個 Trial 帳號沒有自己註冊的正式命名空間，一律要用 **Z** / **Y** 開頭），後面接英數字底線，長度上限通常落在 15~30 碼之間（視系統而定）。> > **ZFE10_NOTE_APP** 這個具體名字純粹是這份講義的命名選擇（**FE10** 對應課程編號方便追溯、**NOTE_APP** 說明這是筆記管理 App），不是規格要求——你可以改成自己喜歡的名字，只要符合上面的命名規則即可。這個名稱跟本機專案資料夾 `fe01_connection_test`、`ui5.yaml` 裡的 `metadata.name`（`fe01connectiontest`）完全是三個獨立的名字，系統不會互相檢查或要求一致，只是這門課的講義寫作習慣讓它們看起來有關係。

1. **Deployment Description**：填一段有意義的描述，例如 **FE10 Deployment Lesson - Note App**
2. **Package**：填 **ZRAPCLOUD**
3. **How do you want to enter Transport Request**：如果有跳出這個提示，選 **Create new**；如果精靈直接跳過（代表這個套件本身是 local、不需要傳輸請求），不用理會
4. 完成後專案資料夾會多出 `ui5-deploy.yaml`，`package.json` 的 `"deploy"` 腳本也會被改寫（原本是佔位符 `"fiori verify"`）
5. 打開 `ui5-deploy.yaml` 核對 `target.authenticationType: reentranceTicket` 這個值有沒有正確帶入（見上一節說明，精靈通常會自動帶對，這一課實測確認不需要額外手動加 `scp` 欄位）

VS Code Step by Step：執行部署

1. 終端機 `cd` 進 `fe01_connection_test` 資料夾

2. 執行 `npm run deploy` (或 Command Palette → **Fiori: Deploy Application**)
3. 部署任務預設是互動式的，會先印出這次部署的摘要 (目標系統 / App 名稱 / 套件 / 傳輸請求) 要求確認，輸入 `y` 或按 Enter 確認
4. 等待終端機印出部署成功訊息——這則訊息通常會直接印出部署後的存取 **URL**，優先用終端機印出的實際網址，不要自己憑印象拼網址。這一課實測的真實輸出 (跟本課一開始猜測的 `/sap/bc/ui5-ui5/...` / 連字號路徑不同，實際是底線 `ui5_ui5`，網域也是 `abap-web.*` 不是 `abap.*`)：

```
``text info abap-deploy-task ZFE10_NOTE_APP * Done * info abap-deploy-task ZFE10_NOTE_APP
App available at https://xxxxxxxx.abap-
web.ap21.hana.ondemand.com/sap/bc/ui5_ui5/sap/zfe10_note_app info abap-deploy-task
ZFE10_NOTE_APP Deployment Successful. ``
```

1. 瀏覽器打開那個網址——⚠️ 這一課第一次實測遇到 **403 Forbidden: The request has been blocked by UCON**，見下一節說明；✅ 這個問題後來已經解開，解法見 **fe11「補充」** 章節，這裡先如實記錄踩坑過程

✅✅ 已解開：**403 Forbidden: blocked by UCON** 不需要管理員權限，開發者自己建一個 **Communication Scenario** 就能解

這裡先如實保留原始踩坑記錄 (部署完第一次拿瀏覽器打開網址，得到 **403 Forbidden / The request has been blocked by UCON**)，但這個問題後來已經完整解開並實測驗證成功，完整過程跟操作步驟寫在 **fe11 講義** (因為是在寫 **fe11** 時回頭補課解決的)：

- 根本原因：**ZFE10_NOTE_APP** 部署完成後，沒有任何 **Communication Scenario** 把它背後的 **OData Service (ZRC08_SB)** 登記進 **UCON** 的允許清單——**UCON (Unified Connectivity)** 是系統在處理請求前，先檢查「這個服務有沒有被明確允許對外開放」的防護層，沒登記就直接 **403**，連登入驗證都不會走到
- ✅ 解法 (開發者層級即可，不需要管理員權限)：在 **Eclipse ADT** 建一個 **Communication Scenario (ZFE10_NOTE_SCENARIO, 套件 ZRAPCLOUD)** → **Inbound 頁籤**加入 **ZRC08_SB** 產生的 **Inbound Service** → 存檔 + 啟用 → 按 **Publish Locally** → 重新整理瀏覽器 → **Notes (1)** 正常顯示，**403** 消失
- 這推翻了 **fe10** 原本「需要管理員權限才能解決」的結論——完整操作 **Step by Step**、以及跟 **ADT「Preview for Fiori Elements App」** (另一條不受 **UCON** 影響、但也不是真正部署網址的路徑) 的差異對照，見 **fe11 講義「補充：回頭解答 fe10 的 UCON 卡點」** 章節，不在這裡重複

驗證方式 (Claude 端)

⚠️ 講義原本規劃的 **I_CustABAPObjDirectoryEntry** 查詢方式，實測在這個環境不可靠——這個 **CDS View** 在這套系統只回傳被標記為「已釋出 API」的極少數物件 (實測總共只有 6 筆，且都不是這一課的部署結果)，查不到 **ZFE10_NOTE_APP**；改直接查 **TADIR** 又被 **ABAP Cloud** 環境擋下「**No authorization to view data**」(延續本課程一貫已知的「**ABAP Cloud** 只能存取 **Released API**」限制)。結論：**Claude** 端目前沒有可靠的獨立查詢方式驗證部署物件是否存在，只能相信 `npm run deploy` 終端機自己回報的成功訊息 (`* Done * / Deployment Successful`)——這也是為什麼「部署成功」與「瀏覽器打得開」要分開驗證：前者已經有終端機輸出當證據，後者才是這次卡住 **UCON** 的地方。

學習目標

- 能講出「部署」跟「`npm start` 本機預覽」的本質差異：後者是暫時的開發輔助，前者才會在 **ABAP** 系統裡留下持久物件

- 知道部署一次會建立四種物件 (BSP App / MIME 資料夾 / ICF Node / Launchpad Descriptor Items)，其中兩種 Eclipse ADT 看不到
- 能講出 `target.authenticationType: reentranceTicket` 的意義，以及這一課實測跟官方通用文件範例 (`target.scp` / 獨立 `credentials` 區塊) 之間的落差——知道要以精靈實際產生的內容為準，不要照書面文件死板套用
- 能操作 **Fiori: Add Deployment Configuration** 精靈 + `npm run deploy`，完成一次端對端部署
- 知道 `I_CustABAPObjDirectoryEntry` 在這個共用 BTP Trial 環境查不到這一課部署的 BSP App (只回傳「已釋出 API」的極少數物件)，也知道直接查 `TADIR` 會被 ABAP Cloud 擋下——理解部署成功與否目前只能相信 `npm run deploy` 終端機自己的回報，不能過度信賴「查得到官方文件提到的某個查詢入口」
- 能講出 UCON (Unified Connectivity) 是什麼：一道「外部請求許可清單」防護機制，在身分驗證之前就先攔截未被明確允許的服務呼叫，**403 blocked by UCON** 不等於部署失敗
- ☒ 知道怎麼解開這個 **403**：開發者在 ADT 建一個 Communication Scenario、加入 Inbound Service、Publish Locally，不需要管理員權限——完整步驟見 fe11 講義 (已實測驗證成功)

物件清單

沒有新增 / 修改任何 CDS View / BDEF / Service Binding——這一課純粹是把 fe01 一路累積的**前端專案**部署到後端，物件清單是部署動作本身產生的：

物件	型別	套件	說明
<code>ZFE10_NOTE_APP</code>	BSP Application	<code>ZRAPCLOUD</code>	<code>fe01_connection_test</code> 專案的部署結果
(MIME 資料夾)	—	<code>ZRAPCLOUD</code>	Eclipse ADT 不可見，只在傳輸請求清單 / <code>I_CustABAPObjDirectoryEntry</code> 看得到
(ICF Node)	SICF	—	Eclipse ADT 不可見，讓瀏覽器能存取這個 BSP App

動手練習

輪到你了：

1. 部署完成後，試著修改 `webapp/i18n/i18n.properties` 裡的 `appTitle`，重新執行一次 `npm run deploy`——確認這是「更新」(同一個 App 名稱) 而不是「建立新物件」，思考一下部署工具怎麼判斷該更新還是新建
2. 打開 `ui5-deploy.yaml` 裡的 `exclude` 欄位說明 (本課 Lecture 沒有示範)，想一想如果專案裡有測試檔案 (`webapp/test/`)，要怎麼設定讓它們不要被打包進部署內容
3. 對照 fe01 用的 `ui5.yaml` (本機預覽用) 跟這一課的 `ui5-deploy.yaml` (部署用)，兩個檔案分別由哪個指令讀取？為什麼要分成兩個檔案而不是共用一個？

驗證方式

使用者在 VS Code 完成 **Fiori: Add Deployment Configuration** 精靈與 `npm run deploy`，終端機印出 **Deployment Successful** 當作部署本身成功的證據 (這是唯一可靠的證據來源——Claude 端嘗試用 `execute_data_query` 查 `I_CustABAPObjDirectoryEntry` / `TADIR` 都無法在這個環境獨立驗證，見上面「驗證方式」段落的實測記錄)；瀏覽器打開部署後的實際 URL 這一步**第一次實測遇到 403 Forbidden**：

blocked by UCON，後來在 **fe11** 補課用 **Communication Scenario + Publish Locally** 解開，重新整理瀏覽器後 **Notes (1)** 正常顯示——端對端驗收完整成功（部署→UCON 允許清單→瀏覽器直接開得到）。

思考題

1. 這一課部署的 App 目前沒有任何 Fiori Launchpad Tile / Intent 設定，只能靠直接貼網址存取——但這次連直接貼網址都被 UCON 擋下來了。你覺得「部署機制本身沒問題、只是最後一哩路的存取權限沒打通」跟「部署根本失敗」，這兩種情況在實務除錯時要怎麼分辨？（提示：想一想這一課怎麼確認「部署成功」跟「瀏覽器打不開」是兩件獨立的事）
2. 這一課實測發現 **ui5-deploy.yaml** 的真實內容跟官方通用文件範例有落差（**scp / credentials** 區塊都沒出現）。如果你是照著網路上一篇教學文章的範例，手動把 **credentials.authenticationType** 那樣的寫法硬加進這次精靈產生的檔案，猜猜看會發生什麼事？（提示：想一想 YAML 檔案裡多一個沒人讀取的區塊，跟少一個工具期待讀到的欄位，兩種情況的後果分別是什麼）
3. **I_CustABAPObjDirectoryEntry** 在這個環境只回傳「已釋出 API」的物件，**ZFE10_NOTE_APP**（一個 UI5 App / BSP 物件）查不到。想一想：官方文件寫「可以用這個 CDS View 查詢部署物件」，跟這一課實測「查不到」，兩者是不是互相矛盾？（提示：官方文件的情境可能是哪種系統 / 哪種物件類型，跟這次的共用 Trial BTP 環境是否完全一樣）

答案

部署動作本身成功（終端機 **Deployment Successful**，App 名稱 **ZFE10_NOTE_APP**，套件 **ZRAPCLOUD**，來源前端專案 **fe01_connection_test / ui5-deploy.yaml**）；瀏覽器直接存取一度被 **403 Forbidden: blocked by UCON** 擋下，後續已在 **fe11** 用 **Communication Scenario (ZFE10_NOTE_SCENARIO) + Publish Locally** 解開，實測驗證成功，**Notes (1)** 正常顯示——不需要管理員權限、不需要完整的 Communication Arrangement / IAM App / Business Role，開發者在 ADT 自己就能解決。